

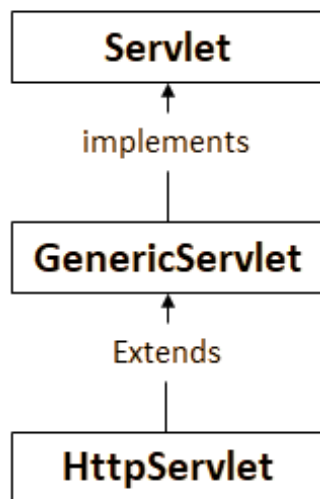
Advance java questions and answers

1. What is Servlet? In how many ways can we create servlet classes?

A **Servlet** is a Java class used to handle client requests and generate dynamic responses in web applications.

Ways to create Servlet:

1. By implementing Servlet interface
2. Extending GenericServlet
3. Extending HttpServlet (most commonly used)



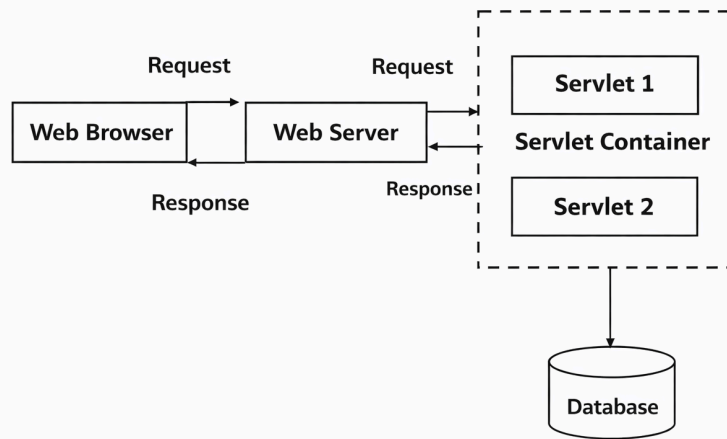
2. Name some Methods of HttpServlet?

- doGet() → Handles GET requests
- doPost() → Handles POST requests
- doPut() → Handles PUT requests
- doDelete() → Handles DELETE requests
- doHead() → Returns headers only
- service() → Dispatches request to appropriate method

3. Explain Servlet Life Cycle and its methods ?

- **Loading** → As per the request particular servlet class will be loaded to container
- **Instantiation** → The web container will create an instance for that loaded class
- **Initialization** → Called once when servlet is created. **init()**
- **Request handling** → Handles every request. **service()**
- **Destruction** → Called once before servlet is destroyed. **destroy()**

Servlet Architecture



4. What is JSP? Life Cycle?

JSP (Java Server Pages) is used to create dynamic web pages using HTML and Java.

Life Cycle:

1. Translation (JSP → Servlet)
2. Compilation
3. Loading
4. Instantiation
5. Initialization → `jspInit()`
6. Request handling → `jspService()`
7. Destruction → `jspDestroy()`

5. Difference between JSP and Servlet?

JSP	Servlet
Used for UI	Used for logic
HTML-centric	Java-centric
Easier to write	More control
Converted to servlet internally	Already Java class
We can java both java and HTML easily by using jsp tags	Writing HTML is difficult, because to write a single tag we should use one <code>print()</code>

6. Explain JSP Tags ?

1. **Directive Tags** (<%@ %>) → Used to import the packages and to handling the exceptions.
2. **Scriptlet Tags** (<% %>) → Used to write java code in jsp
3. **Expression Tags** (<%= %>) → Used to print the values or output
4. **Declaration Tags** (<%! %>) → Used to declare the class members
5. **Action Tags** (<jsp:...>) → Used to control the flow between the pages
6. **Comment Tags** (<%--- ---%>) → Used to comment the code

7. Drawbacks of JSP & Servlet?

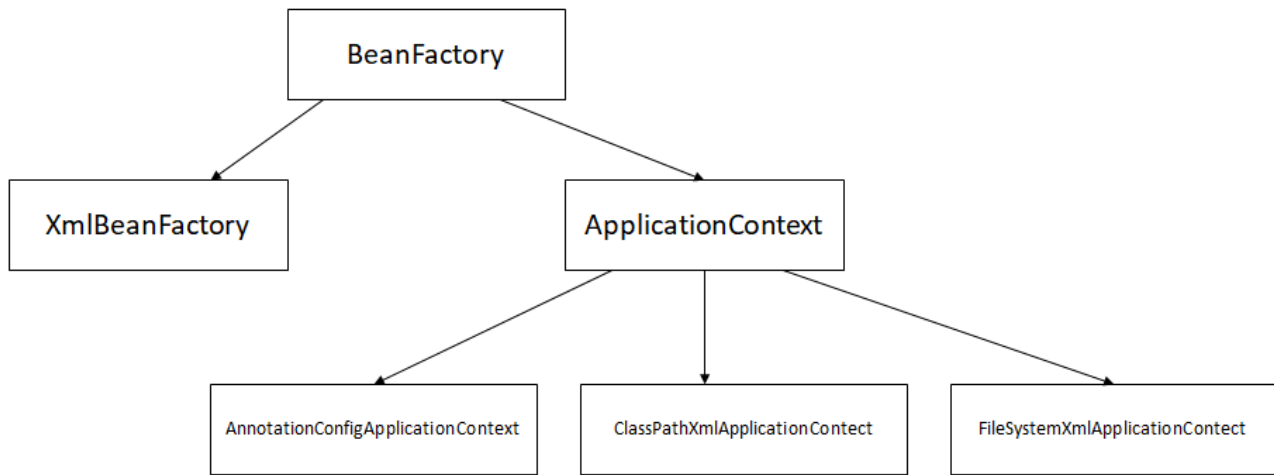
JSP	Servlet
Mixing logic and UI	Boilerplate code
Hard to maintain	Complex to write
Not scalable for large apps	Tight coupling

8. What is Spring?

- **Spring** is a lightweight Java framework used for building enterprise applications with features like DI, AOP, MVC, etc.
- Spring internally works with hibernate
- It allows loose coupling
- Spring default follows Single-ton design pattern, which ensures only one bean is created.
- Spring follows Aspect oriented programming

9. Name Spring Containers?

- **BeanFactory** → Basic container
- **ApplicationContext** → Advanced container (most used)
- **WebApplicationContext** → In **Spring MVC**



10. What is Spring Container?

Spring container is the core part of Spring that creates, manages, and connects objects (beans) in a Spring application.

It is responsible for:

- Creating objects (beans)
- Managing lifecycle
- Injecting dependencies

11. What are Enterprise Applications?

Enterprise applications are large-scale software systems designed to support an organization's business processes and operations. They are scalable, reliable, and used by multiple users across departments.

Example: Banking systems, e-commerce platforms

12. Difference between Spring Container vs Servlet Container

Spring Container	Servlet Container
Manages beans	Manages servlets
Can create beans for user defined classes	creates objects only for servlet classes
Handles DI	Handles HTTP requests
Example: Spring	Example: Apache Tomcat

13. What is Inversion of Control (IOC)?

- Spring IOC stands for Inversion of control, it is a design principle where the control of creating and managing the beans will be managed by spring container
- Object creation is handled by the container instead of the programmer

14. What is Dependency Injection (DI) & Types?

- DI stands for Dependency Injection
- Dependency injection is a design pattern where injecting the objects of one class into another class externally is called as dependency injection
- We can't achieve DI without IOC

Types of dependency injection are:

1. Constructor Injection
2. Setter Injection
3. Variable or Field Injection

15. Name some Spring Core Annotations?

- @Component
- @Autowired
- @Bean
- @Configuration
- @Scope
- @Primary
- @Qualifier

16. Difference between BeanFactory and ApplicationContext?

BeanFactory	ApplicationContext
Basic container in Spring Framework	Advanced container built on BeanFactory
Lazy initialization (beans created when needed)	Eager initialization (beans created at startup)
Limited features	More features (events, internationalization, etc.)
Slower performance (on first use)	Faster at runtime after startup
Used in simple applications	Used in enterprise applications

17. What are Configuration Files?

Configuration Files used to define how objects (beans) are created, configured, and connected in a Spring application.

18. What are the subclasses of ApplicationContext?

- ClassPathXmlApplicationContext
- FileSystemXmlApplicationContext
- AnnotationConfigApplicationContext

19. Explain Spring MVC & and its Manual Configuration?

Spring MVC is a module of the **Spring Framework** that follows the Model-View-Controller architecture to develop web applications by separating data, business logic, and presentation.

Components:

- **Model** → Holds data (business logic, database interaction)
- **View** → Displays data (JSP, HTML pages)
- **Controller** → Handles user requests and controls flow

Manual Configuration in Spring MVC

Manual configuration means setting up Spring MVC without using auto-configuration. It involves:

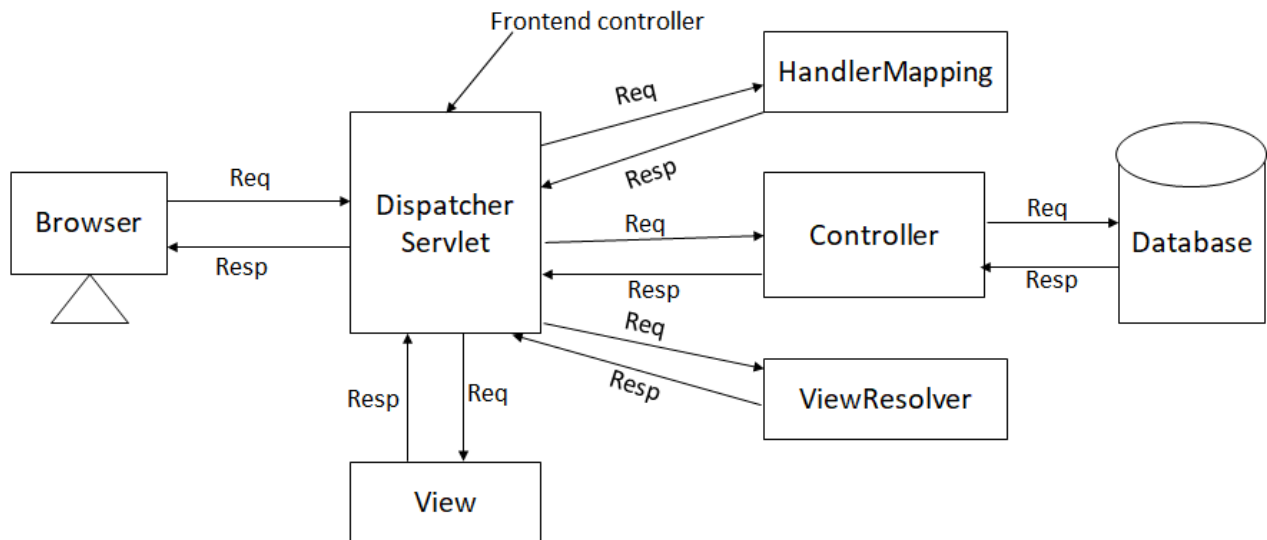
- Configuring **DispatcherServlet** in web.xml to handle all incoming requests
- Creating a Spring configuration file (XML or Java-based) to define beans and enable MVC features
- Enabling component scanning to detect controllers
- Configuring a **View Resolver** to map logical view names to actual JSP pages
- Creating controller classes to handle requests and return views

20. Explain Spring MVC architecture?

Spring MVC, part of the Spring Framework, follows a front controller architecture. When a request is sent from the frontend, it is first received by the **DispatcherServlet**, which acts as the central controller. The DispatcherServlet then consults the **Handler Mapping** to identify the appropriate Controller for the request.

The request is forwarded to the **Controller**, which processes it by interacting with the **Model** (business logic and data). After processing, the Controller returns a **ModelAndView** object to the DispatcherServlet.

The DispatcherServlet then passes the view name to the **View Resolver**, which resolves it into the actual view. Finally, the **View** is rendered and the response is sent back to the client.



21. Explain Handler Mapping & View Resolver?

- **Handler Mapping** is responsible for deciding **which Controller should handle an incoming request**. When a request reaches the DispatcherServlet, it asks the Handler Mapping to find the correct handler based on the URL or request details. The Handler Mapping then returns the appropriate Controller method that should process that request.
- **View Resolver**, on the other hand, is responsible for deciding **which view should be displayed to the user**. After the Controller processes the request, it returns a logical view name (like “home” or “login”). The View Resolver converts this logical name into an actual view file, such as a JSP or HTML page.

22. Difference between spring and spring boot?

Spring	Spring Boot
Manual configuration	Auto configuration
Requires setup	Ready-to-run
Needs external server	Embedded server
Add dependencies manually	Consists of starter files
We can develop web applications	Develop web as well as REST APIs

23. What is Spring Boot?

Spring Boot is a framework which has developed over Spring Framework that is used to develop **standalone, production-ready Spring applications with minimal configuration**.

It simplifies the development process by providing **auto-configuration, embedded servers, and default setups**, so developers don't need to write extensive configuration code.

22. What are starter files? Name some starter files?

In Spring Boot **starter files** are **predefined dependency packages** that simplify project setup.

A **Spring Boot Starter** is a collection of related dependencies that you can include in your project to enable a specific functionality without manually adding each library.

Starter File	Explanation
spring-boot-starter-web	Used to build web applications and REST APIs. Includes Spring MVC, embedded Tomcat, and JSON support.
spring-boot-starter-data-jpa	Used for database access using JPA and Hibernate. Provides repository support and simplifies CRUD operations.
spring-boot-devtools	Provides development-time features like auto restart. Helps in faster development with live reload support.
spring-boot-starter-security	Used to secure applications with authentication and authorization. Provides default login setup and security filters.
spring-boot-starter-thymeleaf	Used for server-side rendering of web pages using Thymeleaf templates. Integrates with Spring MVC for dynamic HTML.
spring-boot-starter-validation	Used for validating user input using annotations like @NotNull, @Size. Ensures data correctness before processing.

24. What are REST APIs & Name some Rules of REST?

REST APIs (Representational State Transfer APIs) are web services that allow communication between client and server using standard HTTP methods. They follow a stateless architecture where data is exchanged in formats like JSON or XML, and each resource is identified by a unique URL.

- Stateless - Each request from the client must contain all the information needed. The server does not store client state between requests.
- Client-Server Architecture - The client and server are separate, allowing independent development and scalability.
- We should send the data in JSON format, because it is lightweight, simple, and JSON data can be directly converted into objects without complex parsing.
- HTTP Methods - For proper HTTP request we should use proper HTTP methods
- The URL path should be independent
- We should follow proper HTTP protocols
- The URL path shouldn't be verb and singular, it should be plural and noun.

24. Explain Spring Boot Thymeleaf?

Spring Boot Thymeleaf is the combination of **Spring Boot** and **Thymeleaf**, where Thymeleaf acts as a server-side template engine to create dynamic HTML pages. It allows Spring Boot applications to render views by binding model data to HTML templates seamlessly.

25. What is Exception Handling and its types in Spring Boot?

Exception Handling in Spring Boot is the process of managing and responding to errors that occur during the execution of a Spring Boot application. It allows the application to handle runtime exceptions gracefully, provide meaningful error messages, and prevent the application from crashing.

Types:

1. **Global Exception Handling**
 - @ControllerAdvice
 - @ExceptionHandler
 - @RestControllerAdvice
2. **Controller Exception Handling**
 - Inside controller
3. **Default error handling**

26. Explain Spring Boot data validation?

Spring Boot Data Validation is the process of **ensuring that the data received from users or clients meets certain rules before being processed or stored**. It helps prevent invalid or malicious data from entering the system, improving reliability and security.

27. Explain server-side rendering and client-side rendering?

Server-Side Rendering (SSR):

Rendering where the **server generates the complete HTML page** and sends it to the client (browser).

Client-Side Rendering (CSR):

Rendering where the **browser generates the UI using JavaScript**, typically after fetching data from the server.

28. What are microservices?

Microservices is an architectural style where an application is divided into **small, independent services**. Each service:

- Runs independently
- Has its own database (optional)
- Communicates via APIs (usually REST)

29. Why use Spring Boot for microservices?

Spring Boot simplifies microservices development by:

- Providing **auto-configuration**
- Embedded servers (Tomcat, Jetty)
- Easy REST API creation
- Integration with Spring Cloud tools

30. How do microservices communicate with each other?

They communicate using:

- **REST APIs (HTTP)**
- Messaging queues (RabbitMQ, Kafka)
- gRPC (less common in Spring Boot)

31. What is service discovery?

Service discovery helps services **find each other dynamically** without hardcoding URLs.

Common tool:

- **Eureka Server**

32. What is an API Gateway?

An API Gateway is a **single entry point** for all client requests.

Responsibilities:

- Routing requests
- Authentication
- Rate limiting

Example:

- Spring Cloud Gateway

33. What is load balancing in microservices?

Load balancing distributes requests across multiple instances of a service.

Tool:

- Spring Cloud LoadBalancer

34. What is fault tolerance?

Fault tolerance ensures the system continues working even if some services fail.

Tools:

- Resilience4j (Circuit Breaker pattern)

35. What is Circuit Breaker pattern?

It prevents repeated calls to a failing service.

States:

- Closed → normal
- Open → block calls
- Half-open → test recovery

36. What is centralized configuration?

Managing configuration for all microservices in one place.

Tool:

- Spring Cloud Config Server

37. What is distributed tracing?

Tracking a request across multiple microservices.

Tools:

- Zipkin
- Sleuth

38. What are challenges in microservices?

- Network latency
- Data consistency
- Deployment complexity
- Monitoring difficulty

39. What is the difference between Monolith and Microservices?

Monolith	Microservices
Single app	Multiple services
Hard to scale	Easily scalable
Tight coupling	Loose coupling
One deployment	Independent deployment

40. What is database per service?

Each microservice has its **own database** to ensure independence.

41. How do you handle security in microservices?

- OAuth2 / JWT authentication
- API Gateway security
- Spring Security integration

42. How would you design an e-commerce system using microservices?

Split into services:

- User Service
- Product Service
- Order Service

- Payment Service

Each communicates via REST or messaging.

43. What happens if one microservice goes down?

Answer:

- Circuit breaker stops calls
- Fallback response is returned
- System continues partially functioning